Elastic Beanstalk

Explore how AWS Elastic Beanstalk manages the deployment and operation of cloud applications by automating infrastructure provisioning, scaling, and health monitoring. Learn to deploy various language platforms using the EB CLI, configure environments, set up IAM roles and environment variables, choose deployment strategies, and enable health dashboards and logging. Understand best practices for rollback, scaling, and advanced configurations to maintain secure, resilient, and scalable applications.

AWS Elastic Beanstalk is a managed service designed to simplify the deployment and operation of applications in the AWS cloud. With just a few commands, it deploys and manages the entire application infrastructure: servers, load balancers, databases, monitoring, and auto-scaling.

Imagine a team that has just wrapped up development on a web application. The deadline to launch is hours away. The app works, the code is tested, but deployment is still unresolved. Provisioning EC2 instances, configuring load balancers, setting up Auto Scaling, defining environment variables, and planning rollback mechanisms—all that infrastructure work remains untouched. There's no time, and no dedicated DevOps team to lean on.

Ask

In this scenario, managing infrastructure manually isn't just inefficient; it risks delaying the entire release. Elastic Beanstalk becomes the most effective solution.

With a single deployment command or console upload, it handles everything: provisioning the environment, scaling resources, routing traffic, monitoring application health, and enabling safe rollbacks.

With a clear understanding of what Elastic Beanstalk is and the kinds of challenges it helps solve, it’s time to explore how the service works in practice. Let’s walk through the deployment workflow and see what Elastic Beanstalk offers.

Deployment workflow

Elastic Beanstalk supports all major languages, such as Java, .NET, Go, PHP, Python, and Ruby, and also works seamlessly with Docker. This flexibility means we can quickly bring our app to the cloud without re-architecting or worrying about low-level configuration.

The application deployment workflow is simple:

- Package the application code and dependencies.
- Upload through the AWS Management Console or the EB CLI.

Beanstalk provisions the necessary infrastructure and manages the deployment lifecycle, including health checks and environment monitoring. Here’s what the workflow looks like:

Ask

Elastic Beanstalk workflow

Environment models

In Elastic Beanstalk, an application runs within an environment, a managed collection of AWS resources with its own configuration and lifecycle. We can create multiple environments for the same application, each hosting a different version or stage (like dev, test, and prod). When launching an environment, we choose one of two tiers:

- **Web server tier:** This tier handles HTTP requests for web applications and APIs. It uses a load balancer and auto-scaling to manage incoming traffic.
- **Worker tier:** This tier runs background jobs or processes asynchronous workloads using Amazon SQS queues. It is ideal for offloading computational or time-consuming tasks from a web application.

This separation allows us to cleanly support real-world application patterns, such as having a web environment for user requests and a worker environment for data processing or scheduled jobs.

Application packaging

Before deploying an application, it's important to understand the structure of a typical Elastic Beanstalk project and the role of its essential files. At a minimum, the deployment package needs to include the application's source code and any dependencies required to run it on the target platform (e.g., `requirements.txt` for Python, `package.json` for Node.js, etc.).

Let's say we want to run a custom script after deployment or tweak some instance settings. We can add a special directory called `.ebextensions` at the root of our project. This directory contains one or more configuration files in YAML or JSON format. These files allow us to:

Ask

- Specify environment variables.
- Add resources such as security groups or S3 buckets.

- Run custom scripts at different lifecycle events.
- Adjust software and instance settings.

index.js C++

```
my-app/
│
├── application.py
├── requirements.txt
└── .ebextensions/
    └── mysettings.config
```

Example of a minimal project structure for a Python web app

Each configuration file inside `.ebextensions` must use the `.config` extension and should be well-formed YAML (or JSON) per AWS documentation. Incorrect formatting or placement can cause deployment failures or unpredictable environment behavior. These configuration files can define option settings, resources, commands, container commands, and more, in sections such as `option_settings`, `resources`, `commands`, and `container_commands`, respectively.

Depending on the chosen platform, one may also include files like `Dockerrun.aws.json` for Docker environments or `Procfile` for custom command execution.

Once the project folders are set up, we can begin the deployments.

Ask

Application deployment with EB CLI

For those who prefer command-line workflows, the EB CLI is a powerful tool that allows us to create, update, configure, and monitor Beanstalk environments from our terminal. It is scriptable and ideal for integrating with CI/CD systems.

To get started with deployments on Elastic Beanstalk, we need to install the EB CLI on the native device, configure the AWS credentials, and then initialize and create the application's environment:

```
# install EB CLI
pip install awsebcli --upgrade --user

# configure AWS credentials
aws configure

# initialize environment
eb init

# create the environment
eb create <environment_name>
```

Install the EB CLI, initialize and create an environment

After the environment is created with all the files of the application, we can deploy the code:

```
eb deploy
```

To access the deployed application, we can use the `eb open` command. This opens the application in the browser.

Here are some additional commands that are helpful while working with Elastic Beanstalk using EB CLI:

Ask

```
# checks logs
eb logs

# check environment status
```

```
# check environment status
eb status

# delete environment
eb scale 3 eb terminate <environment_name>
```

Managing IAM roles and environment variables

Elastic Beanstalk environments often require access to other AWS services such as S3, DynamoDB, or SNS. To enable this securely, IAM roles can be associated with the EC2 instances managed by the environment. Once assigned, these roles allow the application to interact with dependent services without embedding credentials in the code. Roles can be specified during environment creation or updated later using the Elastic Beanstalk console, the EB CLI, or `.config` files under the `.ebextensions` directory.

For application configuration, Elastic Beanstalk supports environment variables at the environment level. These variables can be set or updated using the AWS Management Console, the EB CLI (`eb setenv`), or within configuration files. This method is ideal for passing secrets, environment-specific settings, or deployment-time configuration without hardcoding values into the application.

For even stronger security around secrets and credentials, Elastic Beanstalk can integrate with AWS Secrets Manager or Systems Manager Parameter Store. Applications can reference these secrets at runtime, keeping sensitive values out of source control. The examples below demonstrate how to configure this using the EB CLI and configuration files:

- To add or update an environment variable using the EB CLI:

main.sh

```
eb setenv DATABASE_URL=mysql://user:pass@host/db
```

- To assign a custom IAM instance profile using configuration files, add the following to `.ebextensions`:

index.yaml C++

```
option_settings:
    aws:autoscaling:launchconfiguration:
        IamInstanceProfile: <instance_profile_name>
```

Associate instance profile in .ebextentions file

Application deployment strategies

One of Elastic Beanstalk's key advantages is the ease and safety of updat running applications. When it's time to deploy a new version, we upload our updated code, either through the AWS Management Console, EB CLI, or CI/CD

Ask

pipeline. Beanstalk manages the update/deployment of the new version according to our selected deployment strategy.

Elastic Beanstalk supports the following deployment strategies, each optimized for different needs:

- **All at once:** This updates all instances simultaneously, fast, but can cause downtime if problems occur.
- **Rolling:** This updates instances in batches, minimizing downtime but briefly reducing capacity.
- **Rolling with additional batch:** This maintains full capacity by adding an extra batch during updates.
- **Immutable:** This launches new instances for the update, only switching traffic when healthy, maximizing safety and rollback speed.
- **Blue/green:** This deploys a new environment with the updated version, then switches traffic for zero-downtime cutovers and instant rollback if needed.

Deployment strategies

Health dashboards and logs

Elastic Beanstalk environments provide a dedicated health dashboard, aggregating instance status, deployment events, scaling activity, and real-time environment health in a single view. Each application and instance is evalu continuously, with status clearly indicated using a color-coded scheme (green for healthy, yellow for warnings, red for critical issues, and grey for pending or

Ask

unknown states). This dashboard is often the first place to diagnose the overall status after a deployment or scaling event.

Beyond visual health indicators, Elastic Beanstalk makes all key logs: web server, application, and OS logs, readily accessible. These can be downloaded through the AWS Console or retrieved directly from the command line using `eb logs`. For ongoing visibility and alerting, we can stream logs to Amazon CloudWatch Logs, where they can be retained, searched, and visualized or integrated with automated alarms.

> To monitor Elastic Beanstalk activity within an EC2 instance, logs can be viewed by accessing the `/var/log/eb-activity.log` file. This log shows step-by-step deployment activity, including package installs, `.ebextensions` commands, and app status, which is helpful for troubleshooting failures and verifying configuration changes.

CloudWatch metrics provide deeper operational insight, tracking application and infrastructure health metrics such as CPU utilization, memory usage, disk I/O, and network throughput. Setting up CloudWatch alarms enables us to receive immediate notifications if thresholds are crossed, like increased error rates or high latency, helping our team react before end users are affected. For distributed architectures or applications with complex request flows, enabling AWS X-Ray allows us to trace requests as they move through an environment, pinpointing bottlenecks or failures across services. The illustration gives an overview of how AWS X-Ray and CloudWatch are used to monitor Elastic Beanstalk.

Ask

CloudWatch alarms and X-Ray reports for monitoring

The centralized monitoring, logging, and alerting help us achieve faster troubleshooting, easier compliance, and a stronger operational environment, all essential for production workloads.

Rollback

If an update introduces an error or unexpected behavior, rolling back is straightforward. The Elastic Beanstalk console and EB CLI maintain a history of previously deployed versions, so we can quickly redeploy a known-good version with a few clicks or a single command (`eb deploy --version <label>`). This capability minimizes downtime, reduces risk, and gives teams the confidence to iterate and improve rapidly.

> Best practices include always monitoring application health after deployment, automating rollback triggers for critical failures, and periodically cleaning up unused application versions to avoid hitting storage limits. The illustration below shows a brief overview of how health checks and version histories are managed by Elastic Beanstalk.

Resource scaling

Elastic Beanstalk delivers powerful scaling capabilities, letting our environ

Ask

respond dynamically to workload changes without manual intervention. Each environment is backed by an Auto Scaling group that automatically adjusts the

number of running EC2 instances based on real-time metrics or on a fixed schedule. We can configure minimum and maximum instance counts, define the desired capacity, and set the conditions for scaling, such as target CPU utilization or incoming request rate, either in the AWS Console or within the configuration files.

Scheduled scaling can further optimize our operational costs by increasing capacity during expected peak hours and scaling down when demand drops. Choosing the right EC2 instance types for our workloads also plays a major role in cost management. For example, burstable or ARM-based instances may be more cost-effective for development environments, while production workloads may benefit from compute- or memory-optimized families.

Advanced configuration in Elastic Beanstalk

As applications on Elastic Beanstalk grow, optimizing their architecture for performance and reliability becomes vital. While Application Load Balancer (ALB) is the default for modern web apps, understanding when to adapt the load balancer type or database configuration is key.

Migrating Elastic Load Balancer (ELB) types

ALB excels for most HTTP/HTTPS traffic, offering advanced Layer 7 routing. However, we may migrate to a Network Load Balancer (NLB) when the application requires very high throughput or ultra-low latency layer 4 (TCP/UDP) load balancing.

Elastic Beanstalk simplifies ELB migration. We update the `LoadBalancerType` setting (via Console, saved templates, or `.ebextensions`), and Beanstalk replaces the load balancer while retaining the application's DNS endpoint, minimizing disruption.

Ask

> As a best practice, always test the migration in a staging or preproduction environment first, ensuring the health checks, listener rules, security groups, and routing configurations are all compatible before deploying changes to production.

Decoupling RDS from Elastic Beanstalk

While we can provision an RDS database alongside our application environment for convenience, decoupling the database offers greater data protection and flexibility. The decoupling workflow involves creating a snapshot of the managed RDS instance, restoring it as a standalone RDS database, and then updating the application's connection configuration, usually through environment variables or AWS Secrets Manager, to point to the new, independent instance. Once the migration is validated, we can remove the database resource from Beanstalk management. This separation ensures that critical data persists safely, regardless of changes or rebuilds in an application's environment.

Ask